

Dokumentacja projektu

Nazwa aplikacji: LingoSpark

Autorzy: Wiktor Kycia, Jan Topolewski 4D

Link do repo na github: <https://github.com/wiktorKycia/Flashcards/>

Opis aplikacji

Nasza aplikacja o nazwie „lingoSpark” jest klasyczną aplikacją typu flashcards, w której użytkownicy mogą tworzyć zestawy fiszek i następnie uczyć się z nich języków. Poza mechanizmem fiszek do uczenia się, są dostępne również mechanizmy „dopasowania” oraz „ucz się”. Mechanizm „ucz się” wyróżnia naszą aplikację od innych tego typu, gdyż polega on na rozwiązywaniu zadań generowanych przez LLM-y. W samej aplikacji użytkownicy mają dostępnych dużo dodatkowych funkcji, które pozwalają np. na zmianę hasła, zmianę awatara, polubienie danego quizu lub jego zapisanie.

Opis stron i ich funkcjonalności

- ścieżka ‘/’ – główna podstrona aplikacji. Domyślnie wypisuje listę wszystkich quizów(zestawów fiszek) z bazy danych w kolejności malejącej pod względem ilości „łapek w górę”. Dodatkowo jeżeli w nagłówku wszystkich podstron aplikacji użytkownik wyszuka daną frazę, to zostanie przeniesiony do strony głównej i wtedy zostaną przedstawione wszystkie quizy o nazwie zbliżonej do wyszukanej. Dla użytkowników zalogowanych pokazuje się również lista polubionych quizów.
- ścieżka ‘/user’ – pozwala na zmianę hasła lub innych danych jak adres e-mail, zmianę awatara oraz wylogowanie się
- ścieżka ‘/user/:id’ – w przypadku odwiedzania swojego profilu będąc zalogowanym, użytkownik może zobaczyć listy wszystkich utworzonych przez siebie quizów, polubionych quizów oraz zapisanych quizów. W przypadku odwiedzenia profilu danego użytkownika jednocześnie nie będąc tym użytkownikiem, można zobaczyć tylko utworzone przez niego quizy
- ścieżka ‘/quiz/:id’ – to na tej podstronie można przeglądać fiszki danego quizu. Obsługiwany jest tam także tryb zapamiętywania postępów w nauce poprzez zaznaczenie, co już się umie. Tam także można dać „łapkę w górę” lub „w dół” danemu quizowi, zapisać, skopiować(utworzyć w ten sposób własny quiz). Autor quizu może także usunąć quiz lub przejść do jego edycji.
- ścieżka ‘/quiz/:id/match-challenge’ – jest to zadanie „dopasowania” dla danego quizu polegające na wybieraniu za każdym razem dwóch kafelków, które pochodzą od tej samej fiszki(ich zawartość ma to samo znaczenie, ale w różnych językach)
- ścieżka ‘/quiz/:id/test’ – zawiera na początku ustawienia, w których użytkownik wybiera ile zadań danego typu chce wygenerować(dostępne typy: uzupełnienie luki,

uzupełnienie luki z uzupełnioną już jedną literą, pytania jednokrotnego wyboru).
Zadania te są generowane przez LLM-y dostępne za darmo na GitHub Models

- ścieżka '/register' – zawiera możliwość zarejestrowania nowego konta
- ścieżka '/login' – zawiera możliwość zalogowania się
- Nagłówek – umożliwia wyszukiwanie quizów po nazwach, zmianę motywu, dodawanie quizu(dla zalogowanych) oraz przejście do podstron związanych z zarządzaniem kontem użytkownika

Opis podziału pracy

Podzieliliśmy się pracą w miarę po równo jeśli chodzi o ilość godzin spędzonych nad kodem oraz pod względem trudności

Funkcjonalności wykonane przez każdego z uczestników zespołu:

Wiktor

Konfiguracje dockera

Testy API i komponentów

Backend:

- struktura bazy danych – pomysł, poprawki
- routery (ścieżki podane bez początkowych części typu: /api/jakiśrouter/, po prostu przekopiowane z pliku):
 - (auth.ts) autoryzacja - logowanie i rejestracja użytkownika
 - (quizzesLikesRouter.ts) częściowo router do lajkowania quizów, trasy:
 - PUT /user/:userId(\\d+)/quiz/:quizId(\\d+)
 - DELETE /user/:userId(\\d+)/quiz/:quizId(\\d+)
 - (quizzesProgressRouter.ts) częściowo router do trybu „śledź postępy”, trasy:
 - PATCH /user/:userId(\\d+)/quiz/:quizId(\\d+)/flashcard/:flashcardId(\\d+)
 - PATCH /user/:userId(\\d+)/quiz/:quizId(\\d+)/reset
 - GET /user/:userId(\\d+)/quiz/:quizId(\\d+)
 - (quizzesRouter.ts) częściowo router do quizów, trasy:
 - PUT /:id(\\d+)/flashcards
 - (savedQuizzesRouter.ts) częściowo router do zapisywania quizów, trasy:
 - GET /user/:id(\\d+)/quiz/:id(\\d+)
 - DELETE /
 - (usersRouter.ts) częściowo router do użytkowników, trasy:
 - GET /:userId/avatar
 - POST /avatar
 - PATCH /:id(\\d+)/password

- PATCH /:id(\\d+)
- middleware do autoryzacji
- przesyłanie i zapisywanie plików awatarów

Frontend

- Widok quizu
 - przeglądanie fiszek
 - tryb „śledź postępy”
 - losowanie kolejności fiszek
 - wyświetlanie autora quizu
 - lajkowanie quizów
 - lista fiszek
 - przyciski u góry po prawej (pobierz, zapisz, kopiuj, edytuj, usuń)
 - przycisk do scrollowania w górę
- Tworzenie nowego quizu i widok edycji
 - Wszystkie funkcje
- Przełącznik trybów jasnego i ciemnego
- Logo aplikacji
- Strona profilu użytkownika
 - Lista utworzonych quizów
 - Lista zapisanych quizów
- Ustawienia użytkownika
 - Wszystkie funkcje
- Formularze logowania i rejestracji
- Style

Jan

Backend

- struktura bazy danych – wykonanie, poprawki
- routery:
 - (flashcardsRouter.ts) fiszki

- (quizzesLikesRouter.ts) częściowo router do lajkowania quizów, trasy:
 - GET /:id(\\d+)
 - POST /
 - PATCH /:id(\\d+)
 - DELETE /:id(\\d+)
- (quizzesProgressRouter.ts) częściowo router do trybu „śledź postępy”, trasy:
 - GET /:id(\\d+)
 - POST /
 - PATCH /:id(\\d+)
 - DELETE /:id(\\d+)
- (quizzesRouter.ts) częściowo router do quizów, trasy:
 - GET /
 - GET /:id(\\d+)
 - GET /:id(\\d+)/flashcards
 - POST /
 - PATCH /:id(\\d+)
 - DELETE /:id(\\d+)
- (savedQuizzesRouter.ts) częściowo router do zapisanych quizów, trasy:
 - GET /:id(\\d+)
 - POST /
 - PATCH /:id(\\d+)
 - DELETE /:id(\\d+)
- (tasksGenerationRouter.ts) całość routera do tworzenia zadań przez AI
- (usersRouter.ts) częściowo router do użytkowników, trasy:
 - GET /:id(\\d+)
 - GET /:id(\\d+)/created-quizzes
 - GET /:id(\\d+)/saved-quizzes
 - GET /:id(\\d+)/liked-quizzes
 - GET /:userId(\\d+)/quiz/:quizId(\\d+)
 - POST /
 - DELETE /:id(\\d+)
- middleware do obsługi błędów
- logging w mongo
- prompty do modeli LLM

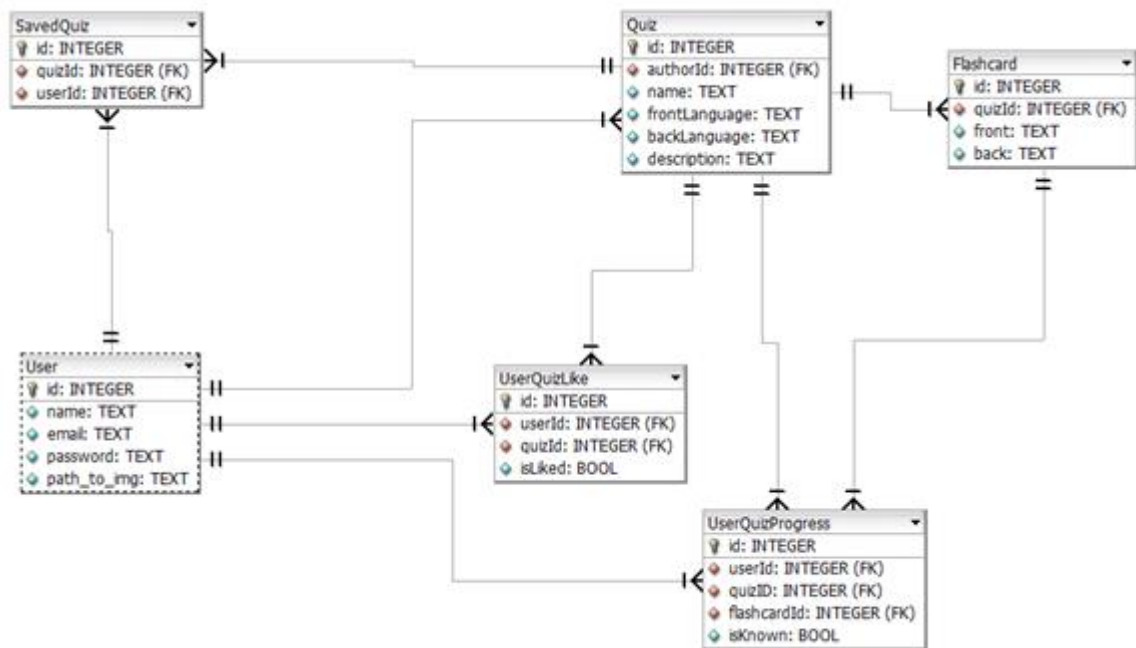
Frontend

- zadania typu “dopasowania”

- zadania typu “ucz się”
- strona główna
- strona profilu użytkownika
 - Lista polubionych quizów
- poprawki styli dla otwarcia aplikacji na urządzeniu mobilnym

Opis i diagram bazy danych

•Główna baza - MySQL



Najważniejszymi elementami bazy są tabele Quiz, Flashcard oraz User. Tabela User jest zabezpieczona przed powtórzeniami w polach name lub email. Wśród dodatkowych tabel występują: SavedQuiz – w celu zapewnienia relacji wiele do wielu i umożliwienia zapisywania konkretnych quizów, podobnie UserQuizLike, ale dodatkowo zawiera opis czy użytkownik dał like (isLiked – true) czy dislike (isLiked – false). Tabela UserQuizProgress jest używana w celu śledzenia postępów w trybie fiszek (isKnown określa znajomość danej fiszki przez danego użytkownika)

• Baza pomocnicza do logów - MongoDB

Używane są 2 kolekcje:

requestLogs -> służy do zapisywania logu o każdym przychodzącym żądaniu

ErrorLogs -> służy do zapisywania informacji o każdym błędzie, który ma zostać w aplikacji obsłużony przez middleware do obsługi błędów

MongoDB Collection: requestLogs

<<document>> RequestLog	
+ timestamp : Date	
+ method : String	
+ url : String	
+ query : Map<String, String>	
+ body : Map<String, Object>	

MongoDB Collection: errorLogs

<<document>> ErrorLog	
+ timestamp : Date	
+ code : String	
+ message : String	

Ważna informacja: Instrukcja uruchomienia aplikacji znajduje się w pliku README.md w repozytorium, a odpowiednie raporty z testów znajdują się w folderach “frontend” oraz “backend”